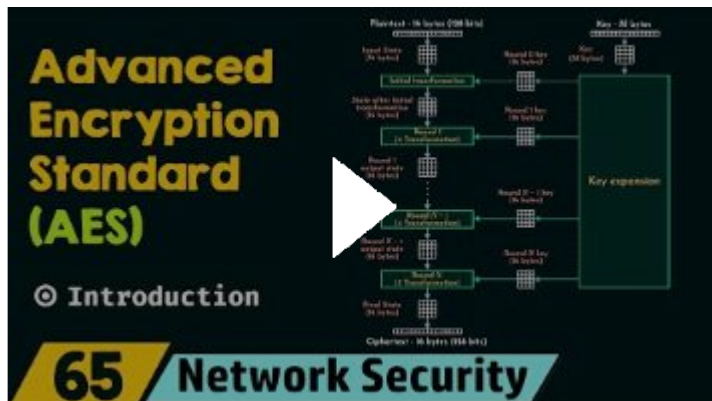


# AES Encryption

jueves, 30 de octubre de 2025

2:57 p. m.

Primer video: [Introduction to Advanced Encryption Standard \(AES\)](#)



Advanced Encryption Standard

Simétrico -> usa la misma clave para encriptar y desencriptar

Toma bloques de bits, no bit por bit

Entra texto plano de 16 bytes y sale como ciphertext también de 16 bytes

El texto plano se guarda en un array de 16 bytes, o una matriz de 4x4

A este se le hace una transformación inicial con esa matriz, que luego atraviesa una ronda 1 que experimenta 4 transformaciones y luego experimenta varias rondas de 4 transformaciones.

Las 4 transformaciones son substitute bytes, shift rows, mix columns, add round key. Esto da un output que se le da a la siguiente ronda, y sigue y sigue.

En la última ronda solo hacemos 3 transformaciones, no hacemos mix columns.

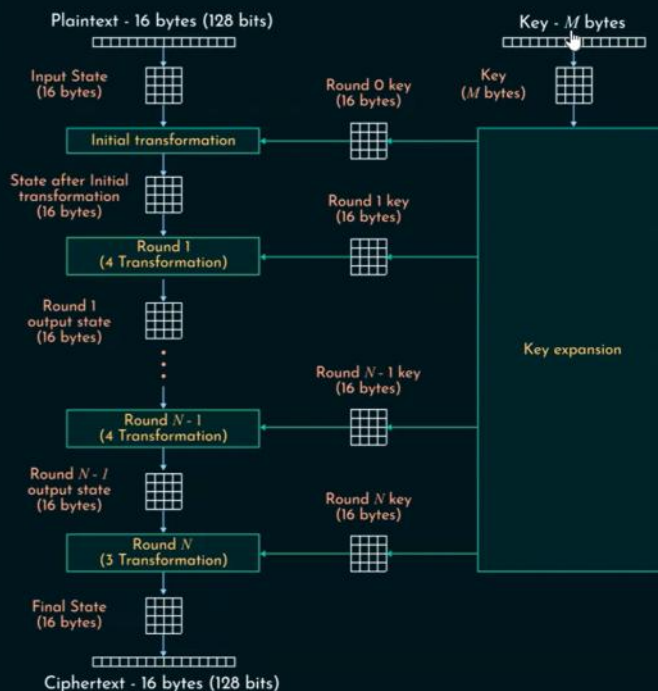
Tras la última ronda, el texto queda de 16 bytes (128 bits) ya cifrado.

Para cada ronda necesitamos una llave, entonces para la transformación inicial necesitamos la key 0, para la primera ronda se requiere la key 1 y así, y cada key es de 16 bytes.

La llave original tiene M bytes, que recibe una key expansion, con una key scheduling algorithm, la cantidad de rondas depende del tamaño de la llave.

# AES Structure

| No. of rounds | Key size (in bits) |
|---------------|--------------------|
| 10            | 128                |
| 12            | 192                |
| 14            | 256                |



nesoacademy.org



AES tiene varias variaciones con distintos parámetros. Existe versión 128, 192 y 256 que son los tamaños de los bits de la key. La principal diferencia es que el número de rondas son de 10, 12 y 14 respectivamente, de resto todas comparten el tamaño del texto plano, y del tamaño de la llave por ronda.

## PROCESO DE ENCRYPTADO Y DESENCRIPTADO

Video: [AES Encryption and Decryption](#)



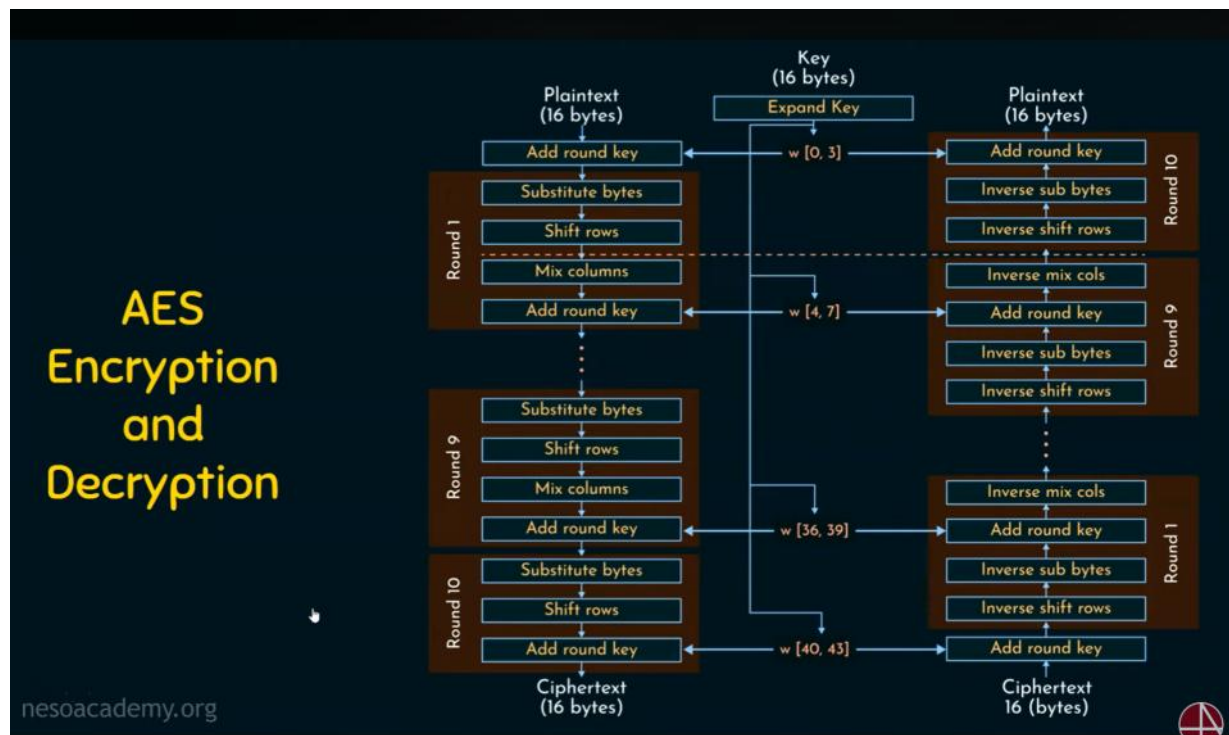
La transformación iniciada no se realiza en una ronda, lo que hace es añadir la round key, por lo que el texto plano se le añade la key, siendo ésta planeada por el key scheduling algorithm, lo que pasa es que se le agregan los 16 bytes de la key a los 16 del texto, lo que da 32.

Luego de esta transformación, ingresa en la primera ronda donde experimenta las 4 transformaciones. En orden son Substitute bytes, shift rows, mix columns y add round key. En la última ronda no se realiza mix columns.

Para seleccionar la llave, se reparte, para la versión de 10 rondas se espera que vaya desde  $w[0, 3]$  hasta  $w[40, 43]$  (no sé de dónde salen estos números, imagino del algoritmo de expansión), y se reparte la llave de a 4 en cada ronda (0, 1, 2 y 3 para la primera, 4, 5, 6, 7 a la segunda, y así hasta 43).  $w$  significa words.

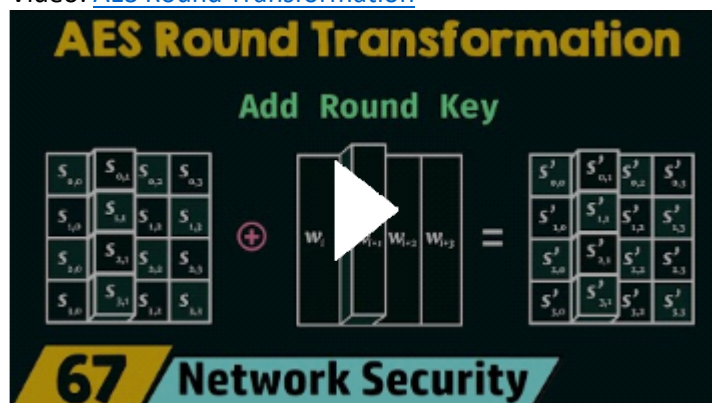
En el caso de la desenscriptación, cogemos el texto cifrado y le sumamos la round key de la ronda 10 de encriptación, cuyo resultado luego pasamos a la ronda 1 donde, en este orden, se realizan los

procesos inverse shift rows, inverse sub bytes, add round key (en la ronda 1 sería la penúltima round key) y por último inverse mix cols. Hacemos esto por todas las rondas hasta la última, en donde no hacemos el último paso de inverse mix cols si no que acabamos con add round key, lo que nos da nuestro texto plano de 16 bytes.



## ROUND TRANSFORMATION

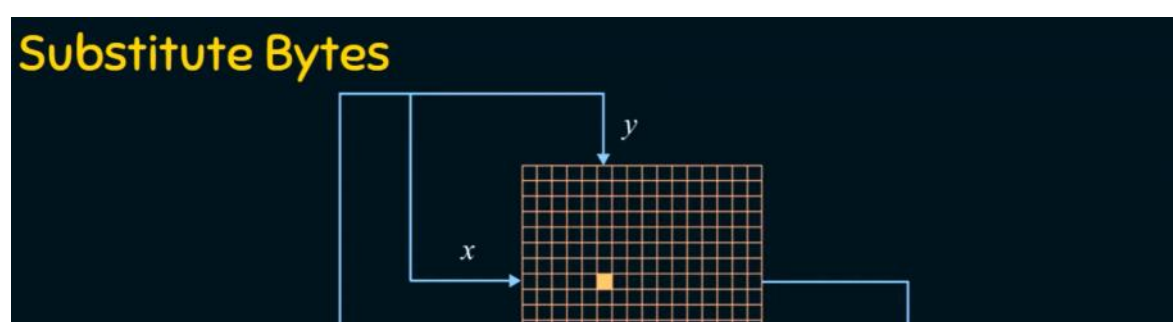
Video: [AES Round Transformation](#)



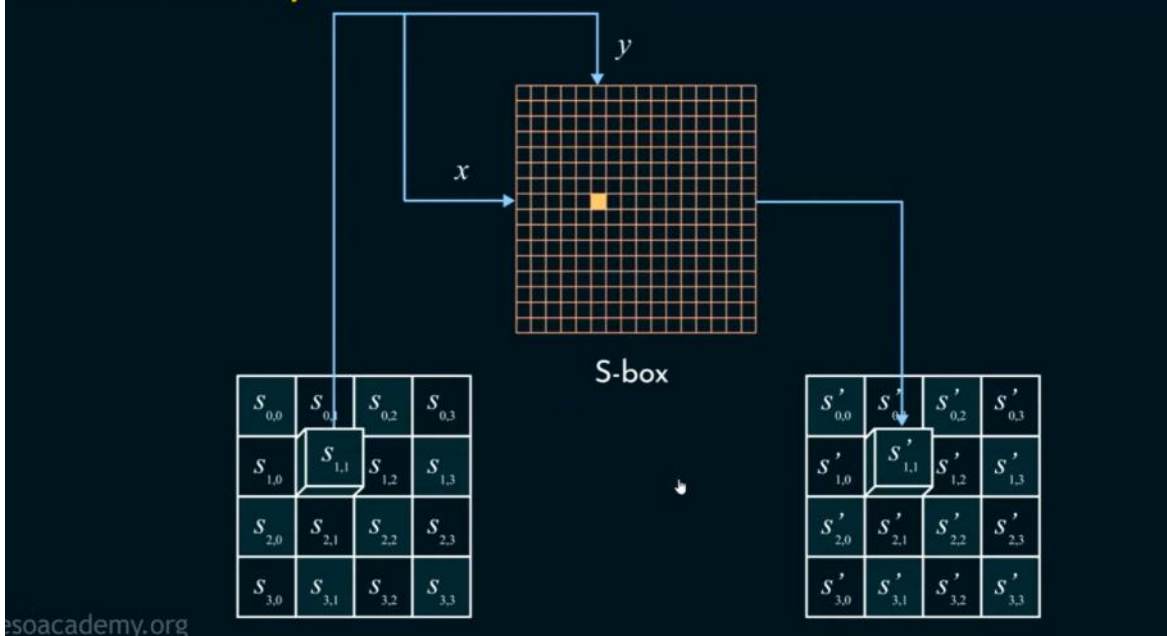
Las funciones de transformación son las siguientes:

### Substitute bytes

Hay una tabla S-box preestablecida que, según la posición del dato que le quiero ingresar (bloque de bits) me dice con qué bits debo reemplazarlo, es una lookup table donde reemplazo unos datos por otros



## Substitute Bytes



### Shift rows

Tenemos los datos organizados en la matriz de 4X4, y vamos a mover las filas que tiene.

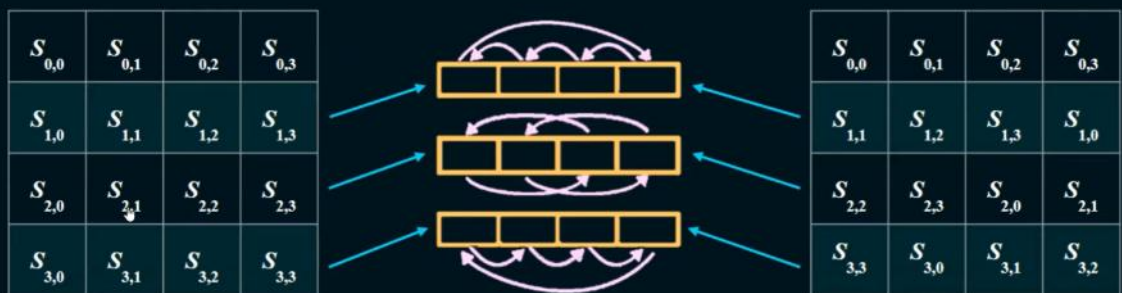
Para la primera fila no le hacemos ningún cambio.

A la segunda fila la vamos a desplazar cada dato una posición hacia atrás, y el dato más a la izquierda se pasa al otro extremo.

La tercera fila lo que hacemos es mover los dos primeros datos, como conjunto, hacia el lado de la derecha de la fila, y los del lado de la derecha los llevamos, en ese orden que tenían, al lado izquierdo, básicamente partir la fila en 2 grupos y moverlos de posición.

Para la última fila lo que hacemos es que desplazamos todos los datos 1 posición a la derecha, y el dato más a la derecha lo dejamos en la primera posición.

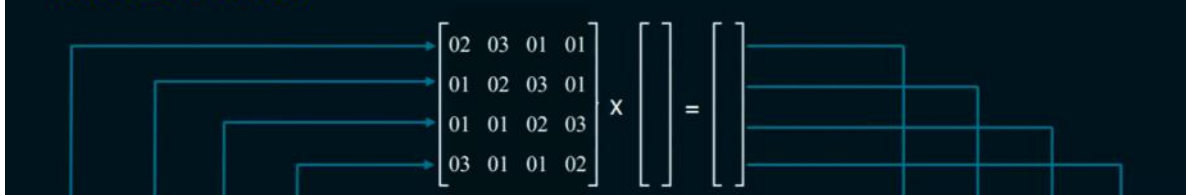
## Shift Rows



### Mix columns:

Lo que hacemos es que tenemos una matriz 4X4, lo que hacemos es multiplicarla por una matriz pre establecida de tal manera que los datos queden revueltos (? Ni idea)

## Mix Columns



## Mix Columns



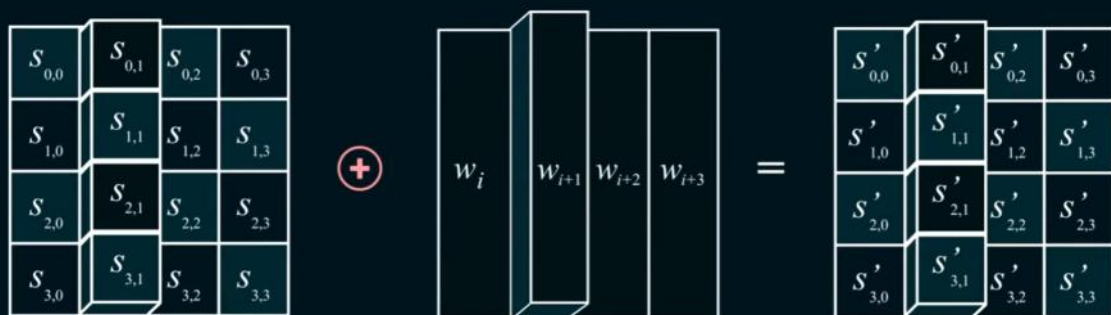
$$\begin{array}{l} [c0] \quad [2 \ 3 \ 1 \ 1] [b0] \\ |c1| = |1 \ 2 \ 3 \ 1| |b1| \\ |c2| \quad |1 \ 1 \ 2 \ 3| |b2| \\ [c3] \quad [3 \ 1 \ 1 \ 2] [b3] \end{array}$$

Fuente: de los deseos <https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/>

Add round key:

Babosa, tienes una matriz y una lista, lo que haces es una suma xor (0 si la suma es 11 o 00, 1 de ser 01 o 10)

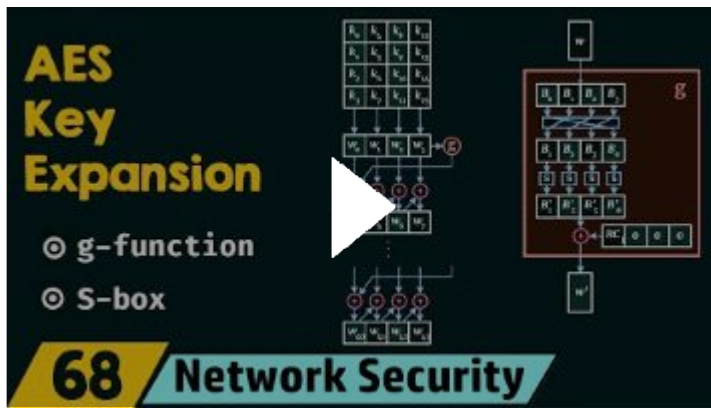
## Add Round Key



KEY EXPANSION

Video: [AES Key Expansion](#)

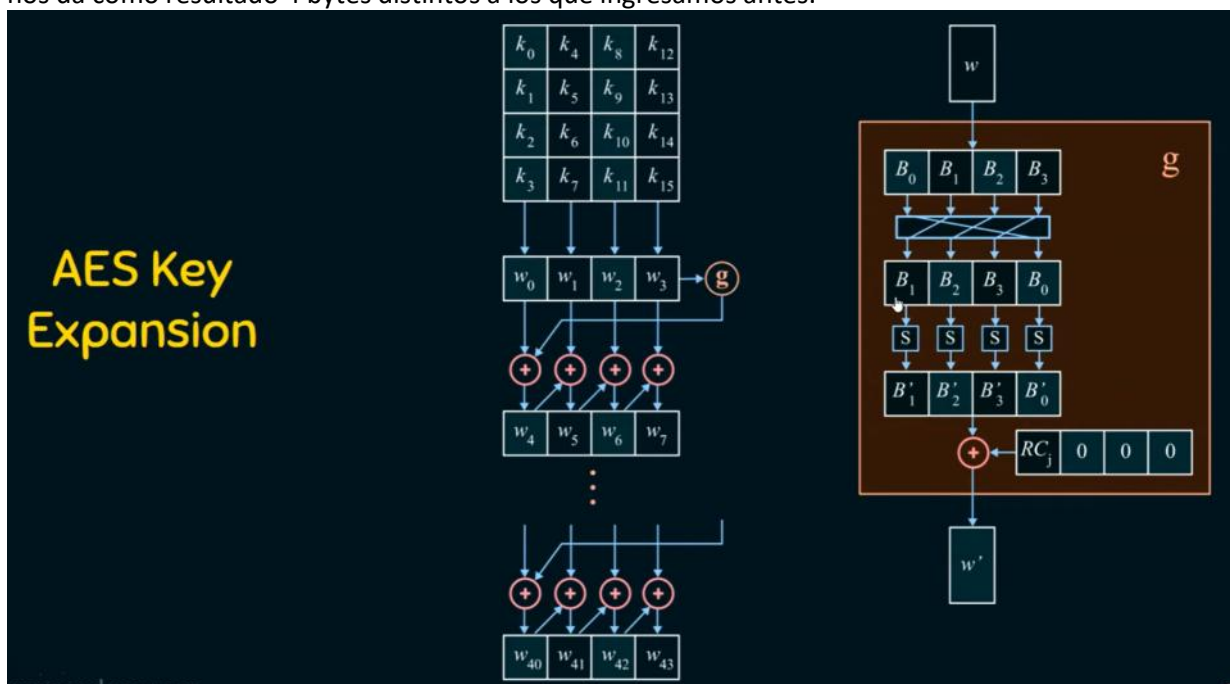




La llave inicial es de 128 bit (16 bytes), por lo que se puede repartir en una matriz 4X4, con cada fila de 32 bits.

La combinación de bytes es una palabra ( $w$  para word), tenemos 4 palabras que son necesarias para la transformación inicial, con las cuales haremos 44 palabras. El byte de la última posición de las filas se pasa a una función  $G$  que se le hace suma xor hacia la primera palabra de la siguiente fila. La primera fila se pasa a la fila de abajo, posición por posición como una suma xor, pero la fila de abajo, tras calcular el valor del primer objeto se le pasa a la segunda posición también como suma xor, ese segundo al calcularse se pasa el tercero, y el tercero al cuarto. Ese cuarto, de nuevo, se pasa por la función  $G$ .

Para la función  $G$ , recibe una palabra (por ejemplo la  $w_3$ ), la cual debe tener 4 bytes internos ( $b_0, b_1, b_2, b_3$ ), luego hacemos un desplazamiento de los elementos hacia la izquierda y el primero se lleva a la última posición. Ya después, pasamos los bytes por la S-box para reemplazar sus valores, lo que nos da como resultado 4 bytes distintos a los que ingresamos antes.



Esta es la tabla S, los primeros 4 bits representan la fila y los últimos representan la columna

## S-Box in AES

|    | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0a | 0b | 0c | 0d | 0e | 0f |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 00 | 63 | 7c | 77 | 7b | f2 | 6b | 6f | c5 | 30 | 01 | 67 | 2b | fe | d7 | ab | 76 |
| 10 | ca | 82 | c9 | 7d | fa | 59 | 47 | f0 | ad | d4 | a2 | af | 9c | a4 | 72 | c0 |
| 20 | b7 | fd | 93 | 26 | 36 | 3f | f7 | cc | 34 | a5 | e5 | f1 | 71 | d8 | 31 | 15 |
| 30 | 04 | c7 | 23 | c3 | 18 | 96 | 05 | 9a | 07 | 12 | 80 | e2 | eb | 27 | b2 | 75 |
| 40 | 09 | 83 | 2c | 1a | 1b | 6e | 5a | a0 | 52 | 3b | d6 | b3 | 29 | e3 | 2f | 84 |
| 50 | 53 | d1 | 00 | ed | 20 | fc | b1 | 5b | 6a | cb | be | 39 | 4a | 4c | 58 | cf |
| 60 | d0 | ef | aa | fb | 43 | 4d | 33 | 85 | 45 | f9 | 02 | 7f | 50 | 3c | 9f | a8 |
| 70 | 51 | a3 | 40 | 8f | 92 | 9d | 38 | f5 | bc | b6 | da | 21 | 10 | ff | f3 | d2 |
| 80 | cd | 0c | 13 | ec | 5f | 97 | 44 | 17 | c4 | a7 | 7e | 3d | 64 | 5d | 19 | 73 |
| 90 | 60 | 81 | 4f | dc | 22 | 2a | 90 | 88 | 46 | ee | b8 | 14 | de | 5e | 0b | db |
| a0 | e0 | 32 | 3a | 0a | 49 | 06 | 24 | 5c | c2 | d3 | ac | 62 | 91 | 95 | e4 | 79 |
| b0 | e7 | c8 | 37 | 6d | 8d | d5 | 4e | a9 | 6c | 56 | f4 | ea | 65 | 7a | ae | 08 |
| c0 | ba | 78 | 25 | 2e | 1c | a6 | b4 | c6 | e8 | dd | 74 | 1f | 4b | bd | 8b | 8a |
| d0 | 70 | 3e | b5 | 66 | 48 | 03 | f6 | 0e | 61 | 35 | 57 | b9 | 86 | c1 | 1d | 9e |
| e0 | e1 | f8 | 98 | 11 | 69 | d9 | 8e | 94 | 9b | 1e | 87 | e9 | ce | 55 | 28 | df |
| f0 | 8c | a1 | 89 | 0d | bf | e6 | 42 | 68 | 41 | 99 | 2d | 0f | b0 | 54 | bb | 16 |

Ya luego del reemplazo con la tabla S sumamos esos valores con xor a una cosa con RCj y otros 3 ceros. El valor RCj se llama Round constant y está predefinido por la ronda que se está haciendo

## Round Constant in AES-128



Follow

@nesoacademy

| Round                                                                             | RC                                  | Round | RC                                  |
|-----------------------------------------------------------------------------------|-------------------------------------|-------|-------------------------------------|
| 1                                                                                 | ( <u>01</u> 00 00 00) <sub>16</sub> | 6     | ( <u>20</u> 00 00 00) <sub>16</sub> |
| 2                                                                                 | ( <u>02</u> 00 00 00) <sub>16</sub> | 7     | ( <u>40</u> 00 00 00) <sub>16</sub> |
| 3                                                                                 | ( <u>04</u> 00 00 00) <sub>16</sub> | 8     | ( <u>80</u> 00 00 00) <sub>16</sub> |
| 4                                                                                 | ( <u>08</u> 00 00 00) <sub>16</sub> | 9     | ( <u>1B</u> 00 00 00) <sub>16</sub> |
| 5                                                                                 | ( <u>10</u> 00 00 00) <sub>16</sub> | 10    | ( <u>36</u> 00 00 00) <sub>16</sub> |
| Note: Initial Transformation takes ( <u>00</u> 00 00 00) <sub>16</sub> as the RC. |                                     |       |                                     |

Estos números, recordar, son bytes.

Este valor RCj se le suma a todos los bytes.